```
// CsvOperations.h

#pragma once

#define NODEMCU_FILE

#define INVENTORY_FILE_TEMP "/temp.csv"

#define BUFFER_LIMIT 64
// The total buffer size of a completely
// packed csv line would actually be 48
// bytes, but 64 allows a safe margin and
// is a round number

#include ".common/comm.h"
#include "SoftwareSerial.h"
#include <CSV_Parser.h>
#include <stdint.h>

#include <SD.h>
#include <SPI.h>

struct InventoryRecordLine
{
  uint32_t reg_date;
  uint32_t expiry_date;
  char item_name[16];
  char box_id[3];
  uint8_t notified;
  bool physical;
  bool supported;
  FoodCondition condition;
  ProfileID preset;
};

void WritePhysicals(SoftwareSerial *sender);
void SaveInventory(InventoryRecordLine *records);
void AddRecord(const InventoryRecordLine *record);
void DeleteRecord(const char *target_box_id);
void CleanCSV();
void UpdateCSV(const char *csv_input);
void CheckAndNotify();

uint16_t InventoryCount();
void WriteInventoryWindow(SoftwareSerial *sender, uint16_t start, uint8_t count);

// CsvOperations.cpp

#include "CsvOperations.h"

#include "NtfyNotifier.h"
#include <Arduino.h>
#include <CSV_Parser.h>
```

```
#include <SD.h>
#include <SPI.h>
#include <SoftwareSerial.h>
#include <cstdint>

#include ".common/comm.h"
#include "WString.h"
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <sys/types.h>

File read_file;
File edit_file;
File copy_file;

inline void InventoryInit()
{
  if (read_file)
    read_file.close();

  if ((read_file = SD.open(INVENTORY_FILE, FILE_READ)))
    d_SerialPrintln("File Exists.");
  else
    d_SerialPrintln("Error! File not found!");
}

// we're going to be using row-by-row indexing to fix memory/buffer overflows
// order is something like this
// 0         1       2          3         4             5         6
// notified, box_id, item_name, reg_date, expiry_date, physical, condition
// uint8_t,  string, string,    ulong,    ulong,       uint8_t,  uint8_t

void WritePhysicals(SoftwareSerial *sender)
{
  InventoryInit();

  char field[32];
  uint8_t field_idx = 0;
  uint8_t col = 0;

  char box_id[16] = {0};
  uint16_t profile = 0;
  bool is_physical = false;

  while (read_file.available())
  {
    char c = read_file.read();

    if (c == ',' || c == '\n' || c == '\r')
    {
      field[field_idx] = '\0';

      if (col == 1)
```

```
{
  // box_id
  strncpy(box_id, field, sizeof(box_id) - 1);
  box_id[sizeof(box_id) - 1] = '\0';
}
else if (col == 5)
  // physical flag
  is_physical = (field[0] == '1');
else if (col == 7)
  // profile enum
  profile = (uint16_t)atoi(field);

field_idx = 0;
col++;

if (c == '\n' || c == '\r')
{
    if (is_physical)
    {
      sender->print(box_id);
      sender->print(',');
      sender->print(profile);
      sender->print(';');
    }

    col = 0;
    profile = 0;
    is_physical = false;
    box_id[0] = '\0';
  }
}
else
{
  if (field_idx < sizeof(field) - 1)
    field[field_idx++] = c;
}
  }

  read_file.close();

  sender->print(MSG_TERMINATOR);
}

void AddRecord(const InventoryRecordLine *record)
{
  d_SerialPrint("Free Heap:");
  d_SerialPrintlnV(ESP.getFreeHeap());

  if ((edit_file = SD.open(INVENTORY_FILE, FILE_WRITE)))
  {
    edit_file.printf("0,%s,%s,%u,%u,%d,%d,%u\n", record->box_id, record->item_name,
record->reg_date,
                     record->expiry_date, record->physical, record->condition, record-
```

```
>preset);
    edit_file.close();
    return;
  }
  d_SerialPrint("Error! File Not found");
}

void DeleteRecord(const char *target_box_id)
{
  File in_file = SD.open(INVENTORY_FILE, FILE_READ);
  if (!in_file)
    return;

  File out_file = SD.open(INVENTORY_FILE_TEMP, FILE_WRITE);
  if (!out_file)
  {
    in_file.close();
    return;
  }

  char line_buffer[BUFFER_LIMIT];

  while (in_file.available())
  {
    size_t len = in_file.readBytesUntil('\n', line_buffer, sizeof(line_buffer) - 1);
    line_buffer[len] = '\0';

    while (len > 0 && (line_buffer[len - 1] == '\n' || line_buffer[len - 1] == '\r'))
      line_buffer[--len] = '\0';

    char *cols[8];
    char *token = strtok(line_buffer, ",");

    int i = 0;
    while (token && i < 8)
    {
      cols[i++] = token;
      token = strtok(NULL, ",");
    }

    if (i < 8)
      continue;

    if (strcmp(cols[1], target_box_id) == 0)
      continue;

    out_file.print(cols[0]); // notified
    out_file.print(',');
    out_file.print(cols[1]); // box_id
    out_file.print(',');
    out_file.print(cols[2]); // item_name
    out_file.print(',');
    out_file.print(cols[3]); // reg_date
```

```
    out_file.print(',');
    out_file.print(cols[4]); // expiry_date
    out_file.print(',');
    out_file.print(cols[5]); // physical
    out_file.print(',');
    out_file.print(cols[6]); // condition
    out_file.print(',');
    out_file.println(cols[7]); // profile
  }

  in_file.close();
  out_file.close();

  SD.remove(INVENTORY_FILE);
  SD.rename(INVENTORY_FILE_TEMP, INVENTORY_FILE);
}
```

```
void UpdateCSV(const char *csv_input)
{
  const int max_updates = 16;

  char update_ids[max_updates][10];
  uint8_t update_conditions[max_updates];
  int update_count = 0;

  char buffer[BUFFER_LIMIT];
  strncpy(buffer, csv_input, sizeof(buffer) - 1);
  buffer[sizeof(buffer) - 1] = '\0';

  char *line = buffer;

  while (*line != '\0' && update_count < max_updates)
  {
    char *next_line = strchr(line, '\n');
    if (next_line)
    {
      *next_line = '\0';
      next_line++;
    }

    size_t line_len = strlen(line);
    while (line_len > 0 && (line[line_len - 1] == '\r' || line[line_len - 1] == '\n'))
      line[--line_len] = '\0';

    if (*line != '\0')
    {
      char *comma = strchr(line, ',');
      if (comma)
      {
        *comma = '\0';
        char *id = line;
        char *cond = comma + 1;
```

```
      strncpy(update_ids[update_count], id, sizeof(update_ids[0]) - 1);
      update_ids[update_count][sizeof(update_ids[0]) - 1] = '\0';
      update_conditions[update_count] = (uint8_t)atoi(cond);
      update_count++;
    }
  }

  if (!next_line)
    break;

  line = next_line;
}

File in_file = SD.open(INVENTORY_FILE, FILE_READ);
if (!in_file)
  return;

File out_file = SD.open(INVENTORY_FILE_TEMP, FILE_WRITE);
if (!out_file)
{
  in_file.close();
  return;
}

char line_buffer[BUFFER_LIMIT];

while (in_file.available())
{
  size_t len = in_file.readBytesUntil('\n', line_buffer, sizeof(line_buffer) - 1);
  line_buffer[len] = '\0';

  while (len > 0 && (line_buffer[len - 1] == '\n' || line_buffer[len - 1] == '\r'))
    line_buffer[--len] = '\0';

  char *cols[8];
  char *token = strtok(line_buffer, ",");
  int i = 0;

  while (token && i < 8)
  {
    cols[i++] = token;
    token = strtok(NULL, ",");
  }

  if (i < 8)
    continue;

  uint8_t notified = (uint8_t)atoi(cols[0]);
  uint8_t current_condition = (uint8_t)atoi(cols[6]);

  for (int j = 0; j < update_count; j++)
  {
    if (strcmp(cols[1], update_ids[j]) == 0)
```

```
{
  uint8_t incoming_condition = update_conditions[j];
  bool should_update = false;

  if (current_condition == UNKNOWN && incoming_condition != UNKNOWN)
  {
    should_update = true;
  }
  else if (incoming_condition > current_condition)
  {
    should_update = true;
  }

  if (should_update)
  {
    current_condition = incoming_condition;
    notified = 0;
    }

    break;
  }
}

out_file.print(notified);
out_file.print(',');
out_file.print(cols[1]); // box_id
out_file.print(',');
out_file.print(cols[2]); // item_name
out_file.print(',');
out_file.print(cols[3]); // reg_date
out_file.print(',');
out_file.print(cols[4]); // expiry_date
out_file.print(',');
out_file.print(cols[5]); // physical
out_file.print(',');
out_file.print(current_condition);
    out_file.print(',');
    out_file.println(cols[7]); // profile
  }

  in_file.close();
  out_file.close();

  SD.remove(INVENTORY_FILE);
  SD.rename(INVENTORY_FILE_TEMP, INVENTORY_FILE);
}

FoodCondition WorseCondition(FoodCondition a, FoodCondition b)
{
  if (a == UNKNOWN)
    return b;
  if (b == UNKNOWN)
    return a;
```

```
  return (a > b) ? a : b;
}

void CheckAndNotify()
{
  File in_file = SD.open(INVENTORY_FILE, FILE_READ);
  if (!in_file)
    return;

  SD.remove(INVENTORY_FILE_TEMP);
  File out_file = SD.open(INVENTORY_FILE_TEMP, FILE_WRITE);
  if (!out_file)
  {
    in_file.close();
    return;
  }
char line_buffer[BUFFER_LIMIT];
uint32_t now = (uint32_t)time(nullptr);

while (in_file.available())
{
  size_t len = in_file.readBytesUntil('\n', line_buffer, sizeof(line_buffer) - 1);
  line_buffer[len] = '\0';

  while (len > 0 && (line_buffer[len - 1] == '\n' || line_buffer[len - 1] == '\r'))
    line_buffer[--len] = '\0';

  char *cols[8];
  char *token = strtok(line_buffer, ",");
  int i = 0;

  while (token && i < 8)
  {
    cols[i++] = token;
    token = strtok(NULL, ",");
  }

if (i < 8)
  continue;

uint8_t notified = (uint8_t)atoi(cols[0]);
const char *box_index = cols[1];
const char *item_name = cols[2];
uint32_t reg_date = (uint32_t)atol(cols[3]);
uint32_t expiry = (uint32_t)atol(cols[4]);
bool physical = atoi(cols[5]) != 0;
FoodCondition stored_condition = (FoodCondition)atoi(cols[6]);
const char *preset = cols[7];

FoodCondition time_condition = FRESH;

if (expiry <= now)
```

```
    {
      time_condition = EXPIRED;
    }
    else if (expiry > reg_date && now >= reg_date && now <= expiry)
    {
      uint32_t total_life = expiry - reg_date;
      uint32_t remaining_life = expiry - now;

      if (total_life > 0)
      {
        uint32_t percent_remaining = (uint32_t)(((uint64_t)remaining_life * 100ULL) /
(uint64_t)total_life);

        if (percent_remaining <= 30)
          time_condition = GOING_BAD;
      }
    }

    FoodCondition final_condition = WorseCondition(stored_condition, time_condition);

    if (final_condition > stored_condition)
      notified = 0;

    if (notified == 0 && final_condition != UNKNOWN && final_condition != FRESH)
    {
      switch (final_condition)
      {
        case GOING_BAD:
          {
            String msg = String("*") + item_name + "* in box *" + box_index + "* is
going bad.";
            NtfyNotify("warning", "Item Going Bad", msg.c_str(), "high");
            notified = 1;
          }
          break;

        case EXPIRED:
          {
            String msg = String("*") + item_name + "* in box *" + box_index + "* has
expired!";
            NtfyNotify("warning,skull", "Item Expired!", msg.c_str(), "urgent");
            notified = 1;
          }
          break;

        default:
          break;
      }
    }

    out_file.print(notified);
    out_file.print(',');
    out_file.print(box_index);
```

```
  out_file.print(',');
  out_file.print(item_name);
  out_file.print(',');
  out_file.print(reg_date);
  out_file.print(',');
  out_file.print(expiry);
  out_file.print(',');
  out_file.print(physical ? 1 : 0);
  out_file.print(',');
  out_file.print((uint8_t)final_condition);
  out_file.print(',');
  out_file.println(preset);
}

in_file.close();
out_file.close();

  SD.remove(INVENTORY_FILE);
  SD.rename(INVENTORY_FILE_TEMP, INVENTORY_FILE);
}

uint16_t InventoryCount()
{
  d_SerialPrintln("InventoryCount: begin");

  InventoryInit();

  if (!read_file)
  {
    d_SerialPrintln("InventoryCount: read_file invalid");
    return 0;
  }

  uint16_t count = 0;
  bool has_data = false;

while (read_file.available())
{
  char c = read_file.read();

  if (c != '\n' && c != '\r')
    has_data = true;

  if ((c == '\n' || c == '\r') && has_data)
  {
    count++;
    has_data = false;

    if (c == '\r' && read_file.available())
    {
      char next = read_file.peek();
      if (next == '\n')
        read_file.read();
```

```
      }
    }
  }

  if (has_data)
    count++;

  read_file.close();

  d_SerialPrint("InventoryCount: final count = ");
  d_SerialPrintln(count);

  return count;
}

void WriteInventoryWindow(SoftwareSerial *sender, uint16_t start, uint8_t rows)
{
  InventoryInit();
  if (!read_file)
  {
    sender->print("IVW,");
    sender->print(MSG_TERMINATOR);
    return;
  }

  sender->print("IVW,");

  char field[40];
  uint8_t field_idx = 0;
  uint8_t col = 0;

  char item_name[32] = {0};
  char box_id[16] = {0};
  char condition[8] = "3";

  uint16_t row_index = 0;
  uint8_t sent = 0;
  bool first_row = true;

  while (read_file.available() && sent < rows)
  {
    char c = read_file.read();

    if (c == ',' || c == '\n' || c == '\r')
    {
      field[field_idx] = '\0';

      if (col == 1)
      {
        strncpy(box_id, field, sizeof(box_id) - 1);
        box_id[sizeof(box_id) - 1] = '\0';
      }
      else if (col == 2)
```

```
{
  strncpy(item_name, field, sizeof(item_name) - 1);
  item_name[sizeof(item_name) - 1] = '\0';
}
else if (col == 6)
{
  strncpy(condition, field, sizeof(condition) - 1);
  condition[sizeof(condition) - 1] = '\0';
}

field_idx = 0;
col++;

if (c == '\n' || c == '\r')
{
  if (col > 1)
  {
    if (row_index >= start)
    {
      if (!first_row)
        sender->print('\n');

      sender->print(item_name);
      sender->print('|');
      sender->print(condition);
      sender->print('|');
      sender->print(box_id);

      first_row = false;
      sent++;
    }

    row_index++;
  }

  col = 0;
    item_name[0] = '\0';
    box_id[0] = '\0';
    strcpy(condition, "3");

    if (c == '\r' && read_file.available())
    {
      char next = read_file.peek();
      if (next == '\n')
        read_file.read();
    }
  }
}
else
{
  if (field_idx < sizeof(field) - 1)
    field[field_idx++] = c;
}
```

```
    }

    read_file.close();
    sender->print(MSG_TERMINATOR);
}
```